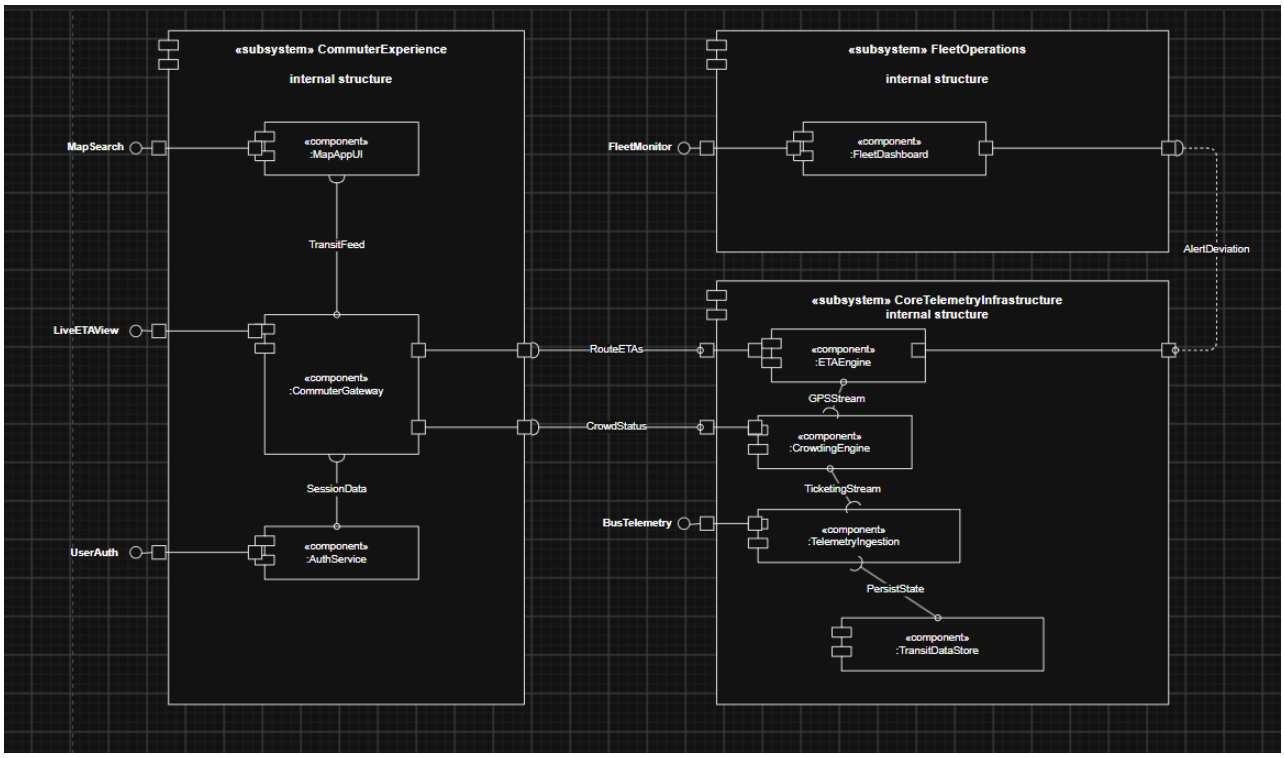
Software Engineering Lab-3

Public Bus Live Tracking and Crowding Estimator

PES2UG24CS461

Component Diagram:



1. Architectural Choice

I selected a Microservices Architecture divided into three main subsystems based on the system requirements and project epics:

- **Commuter Experience Subsystem:** Contains the Map App UI, Commuter Gateway, and Auth Service to handle stop searches, live map tracking, and user sessions (PBLT-4, PBLT-5).
- **Fleet Operations Subsystem:** Contains the Fleet Dashboard to help transit controllers monitor bus locations, schedules, and route alerts (PBLT-8).
- **Core Telemetry Infrastructure Subsystem:** Contains Telemetry Ingestion, the ETA Engine, the Crowding Engine, and the Transit Data Store to ingest live vehicle feeds, calculate stop arrival times, and estimate passenger crowding (PBLT-6, PBLT-10).

These subsystems communicate across dedicated boundary ports using standard provided and required interfaces.

2. Two Reasons for Selection

- **Separating Heavy Bus Data Ingestion from Commuter Requests:**

The system must continuously receive GPS coordinates and passenger boarding counts from up to 1,000 buses every 5 seconds (FR-001, NFR-001). In a traditional Layered or Client-Server architecture, saving this high-volume stream into a single central database creates lock contention and slows down commuters trying to view stop arrival times (FR-003). Microservices place data ingestion into an isolated service, ensuring continuous vehicle writes never slow down commuter read traffic.

- **Independent Scaling and Fault Isolation:**

Recalculating dynamic arrival times (FR-001) and detecting route deviations greater than 200 meters (FR-005) requires significant processing power. With microservices, I can scale compute resources for the ETA Engine during morning and evening rush hours without having to scale the rest of the application. Furthermore, if the Crowding Engine encounters issues with ticketing sensor data (FR-002), the remaining services stay online so commuters can still track bus positions without experiencing a system crash.

3. Security Advantage

The architecture protects the system by placing an API Gateway between external users and internal services. The database and calculation services run inside an isolated private network with no direct exposure to the public internet. Transit buses upload their GPS and passenger counts through a secure telemetry port using mutual TLS (mTLS) and hardware authentication, preventing unauthorized devices from injecting fake vehicle data. Commuters and fleet controllers authenticate through the Gateway using secure tokens (JWT), protecting internal services from unauthorized access.

4. Performance Benefit

Microservices allow individual components to communicate using fast, lightweight protocols like gRPC and MQTT alongside an in-memory Redis cache. This ensures that telemetry feeds from 1,000 active buses are processed in under 2 seconds, satisfying NFR-001. Instead of having commuter mobile apps reload and poll the server every few seconds, the Gateway pushes live bus positions and route alerts directly over WebSockets, minimizing network bandwidth and keeping map countdowns updating smoothly every 5 seconds.